

جزوه امتحانی — Big Data (Big Data Analytics)

استاد: دکتر فرساد زمانی بروجنی | امتحان: جزوه باز — ۹۰ دقیقه | منبع: لکچرهای ترم + نمونه سوالات + MMDS

روش پاسخ: تعریف یک خطی ← فرمول داخل باکس ← مثال عددی کوتاه. اول **فهرست صفحات** و **چیت شیت** را ببین. اگر سوال شبیه نمونه استاد بود → بخش 10.

فهرست صفحات (شماره واقعی همین PDF) [INDEX]

در امتحان برو به **شماره صفحه** ستون آخر. این شماره‌ها با پاصفحه PDF یکی هستند.

کلمه کلیدی سوال	بخش	چه بنویسی	صفحه
چیت شیت و فرمول‌های حیاتی	چیت شیت	تعریف + فرمول	1
Definition همه Technical Term ها	Technical Glossary	Definition + Purpose	2
TF-IDF / Collaborative Filtering	بخش 1	فرمول + مثال عددی	6
Cluster ↔ RDBMS / Hadoop	بخش 2	جدول تفاوت‌ها	8
MapReduce / Combiner / Pregel	بخش 3	۳ مرحله + خرابی	11
Join / Relational Algebra	بخش 4	مثال Join	13
Matrix × Vector	بخش 5	Map/Reduce + عدد	13
Shingle / Jaccard / MinHash / LSH / PCY	بخش 6	تعریف + فرمول + مثال	16
Distances / Nominal / Binary	بخش 7	فرمول + مثال	18
Apriori / Closed / Maximal	بخش 8	M ⊆ C ⊆ F + Pass ها	20
Clustering / K-Means / DBSCAN / BFR	بخش 9	مثال یک تکرار	24
حفظیات کامل KDD / مخازن / Hadoop	تکمله A	تعریف‌های جزئی	9
اجرای داخلی / هزینه / Multi-way Join	تکمله B	جزئیات MapReduce	14
MinHash / LSH / CHARM با جزئیات	تکمله C	اثبات و دام‌ها	21
Algorithms Clustering کامل	تکمله D	فرض/مراحل/مقایسه	25
حل نمونه سوالات استاد	بخش 10	پاسخ آماده	28

کل صفحات PDF: 30 | چیت شیت → ص 1 | بخش 1 → ص 6 · بخش 2 → ص 8 · بخش 3 → ص 11 · بخش 4 → ص 13 · بخش 5 → ص 13 · بخش 6 → ص 16 · بخش 7 → ص 18 · بخش 8 → ص 20 · بخش 9 → ص 24 · بخش 10 → ص 28

چیت شیت سریع [CHEAT]

مفهوم	تعریف یک خطی (همین را بنویس)
Shingle	دنباله k حرف/کلمه پشت سرهم در سند
Jaccard	اندازه اشتراک دو مجموعه ÷ اندازه اجتماع
MinHash	احتمال برابر شدن hash دو مجموعه = Jaccard
LSH	فقط جفت‌های احتمالاً مشابه را candidate می‌کند
Collaborative Filtering	توصیه بر اساس کاربران یا آیتم‌های شبیه
Closed itemset	Frequent: هیچ ابرمجموعه‌ای با همان support ندارد
Maximal itemset	Frequent: هیچ ابرمجموعه Frequent ی ندارد
MapReduce	پردازش موازی با Map و Reduce + Fault Tolerance

$$TF_{ij} = f_{ij} / \max_k(f_{kj})$$

$$IDF_i = \log_2(N / n_i)$$

$$TF-IDF = TF_{ij} \times IDF_i$$

که در آن:

f_{ij} = تعداد تکرار کلمه i در سند j

$\max_k(f_{kj})$ = بیشترین تکرار هر کلمه در همان سند j

N = تعداد کل اسناد

n_i = تعداد اسنادی که کلمه i در آن‌ها آمده

$$J(A, B) = |A \cap B| / |A \cup B|$$

$$\Pr[h\pi(C_1) = h\pi(C_2)] = J(C_1, C_2)$$

$$P(\text{candidate}) = 1 - (1 - s^r)^b$$

$$L_2 = \sqrt{\sum (x_i - y_i)^2} \quad L_1 = \sum |x_i - y_i|$$

$$L_\infty = \max |x_i - y_i| \quad \text{Nominal: } d = (p - m) / p$$

$$\text{support}(A \Rightarrow B) = \text{support}(A \cup B)$$

$$\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

$$M \subseteq C \subseteq F$$

$$\text{Map: } (i, m_{ij} \times v_j) \quad \text{Reduce: } u_i = \sum_j (m_{ij} \times v_j)$$

مقایسه	نکته طلایی
RDBMS ↔ Cluster	عمودی/ACID ↔ افقی/commodity و انتقال کد به داده
K-Means ↔ K-Medoids	میانگین (حساس به پرت) ↔ نقطه واقعی داده
Combiner	فقط اگر Reduce انجمنی و جایابی‌پذیر باشد

Technical Glossary — Definition همه اصطلاحات [GLOSS]

این Glossary قبل از درس‌ها آمده تا هر اصطلاح فنی ابتدا Definition داشته باشد. در متن جزوه نیز نام‌های فنی به English نوشته شده‌اند.

Data and Mining Concepts

What it does / When to use	Definition	Technical Term
مسئله‌هایی که Scale آن‌ها به Storage و Processing توزیع شده نیاز دارد.	داده‌ای که حجم، سرعت یا تنوع آن از توان ابزارهای معمول برای پردازش در زمان معقول بیشتر است.	Big Data
کشف رابطه، گروه، Class، Trend و Outlier.	فرایند استخراج Pattern یا Knowledge مفید از حجم زیاد داده.	Data Mining
وقتی Cleaning تا Presentation باید به صورت end-to-end انجام شود.	فرایند کامل تبدیل Raw Data به Knowledge؛ Data Mining یکی از مرحله‌های آن است.	KDD
بهبود کیفیت Input پیش از تحلیل.	تشخیص و اصلاح/حذف Noise، Error، Missing و Inconsistent Data.	Data Cleaning
ساخت Dataset واحد از چند Database/File.	ترکیب Data از چند Source در یک View سازگار.	Data Integration
کم کردن Data غیرضروری.	انتخاب رکوردها و Attribute‌های مرتبط با Question.	Data Selection
آماده‌سازی Data برای Algorithm.	تغییر Format/Scale/Structure مثل Normalization و Aggregation.	Data Transformation
حذف Pattern‌های ضعیف یا تصادفی.	سنجش Validity، Novelty و Usefulness الگو.	Pattern Evaluation
قابل فهم کردن خروجی برای User.	ارائه نتیجه با Rule، Table، Chart یا Visualization.	Knowledge Presentation
Spam detection، تشخیص بیماری، Class مشتری.	Supervised Learning برای پیش‌بینی Label از قبل تعریف شده.	Classification
Document و Customer segmentation و grouping.	Unsupervised Learning برای ساخت Clusterهای طبیعی بدون Label.	Clustering
Fraud، Fault و رخداد نادر مهم.	شیئی که رفتار آن با Pattern عمومی Data سازگار نیست.	Outlier
کاهش پیچیدگی و آماده‌سازی برای Model.	ساخت یا انتخاب نمایش‌های informative از Raw Data.	Feature Extraction
رتبه‌بندی Web Page و تحلیل Graph.	Graph-ranking Algorithm که Importance هر Vertex را از Link‌های ورودی Vertex‌های مهم می‌گیرد.	PageRank
Text Classification و Baseline سریع.	Probabilistic Classifier مبتنی بر Bayes و فرض Conditional Independence Feature‌ها.	Naive Bayes
High-dimensional Classification.	Classifierی که Hyperplane با Maximum Margin بین Class‌ها پیدا می‌کند.	SVM
Classification/Regression ساده بدون Training صریح.	Instance-based Method که Label را از k Neighbor نزدیک می‌گیرد.	KNN
Search و Lookup، Partitioning و کاهش.	تابعی که Key را به Bucket Number نگاشت می‌کند.	Hash Function
بهبود Query و Data Access.	Data Structure کمکی برای یافتن سریع Record بدون Full Scan.	Index

What it does / When to use	Definition	Technical Term
Data ساخت‌یافته و Transaction دقیق.	Database جدولی با Relation، Schema و معمولاً ACID Transaction.	RDBMS
Fault و Horizontal Scaling، Parallelism، Tolerance.	اجرای یک Workload روی مجموعه‌ای از Node‌های متصل.	Cluster Computing
پردازش فایل‌های بسیار بزرگ با HDFS و MapReduce.	Framework متن‌باز برای Distributed Storage و Batch Processing روی Commodity Hardware.	Hadoop
Storage مقاوم به Failure برای File‌های بزرگ.	Distributed File System که File را Block بندی و روی چند DataNode Replicate می‌کند.	HDFS
تضمین Reliability و Correctness در RDBMS.	Atomicity، Consistency، Isolation و Durability؛ چهار اصلی Transaction Property.	ACID
Always-on و Critical System‌های.	در دسترس ماندن Service با حداقل Downtime حتی هنگام Failure.	High Availability
Cluster و Distributed System.	توان ادامه کار درست یا Recovery در حضور Hardware / Software Failure.	Fault Tolerance
Recovery و Availability، Read Scalability.	نگهداری چند Copy از Data روی Node/Location‌های متفاوت.	Data Replication
استفاده متعادل از Resource‌ها.	توزیع Workload بین Node‌ها برای جلوگیری از Bottleneck.	Load Balancing
مدیریت Namespace و محل Block File‌ها.	Master نگهدارنده Metadata و Block Location در HDFS.	NameNode
Heartbeat و ارسال Read/Write Block.	Worker ذخیره‌کننده Block واقعی HDFS.	DataNode
Random Read/Write روی Table‌های بسیار بزرگ.	Distributed Wide-column Database روی HDFS.	HBase
ETL و Analytics روی Data حجیم.	Data Warehouse Layer با Query Language شبیه SQL روی Hadoop.	Hive
نوشتن Pipeline‌های ETL با Script ساده‌تر از MapReduce خام.	High-level Data-flow Platform روی Hadoop.	Pig
اجرای Portable Process‌های Hadoop.	Runtime اجرای Java Bytecode با Memory و Process Isolation.	JVM
Reporting و BI، OLAP.	Repository یکپارچه، Historical و Subject-oriented برای Analytics.	Data Warehouse
Online Analytics و Real-time Monitoring.	Sequence پیوسته و بالقوه نامتناهی از Event‌ها با امکان Scan محدود.	Data Stream
Fault-tolerant و موازی Batch Processing.	Programming Model توزیع‌شده با Map، Shuffle و Reduce.	MapReduce
تضمین می‌کند Key‌های یکسان یک‌جا پردازش شوند.	تعیین Reducer مقصد برای هر Key، معمولاً با Hash.	Partitioning
کاهش Network Traffic برای Operation‌های Associative/Commutative.	Reduce محلی اختیاری در سمت Mapper.	Combiner
Pipeline‌های پیچیده‌تر از Map→Reduce.	DAG از چند Processing Function/Job.	Workflow System
اجرای SuperStep و Recovery با Checkpoint.	Vertex-centric Framework برای Graph Algorithm‌های Iterative.	Pregel

Similarity and Frequent Pattern Concepts

What it does / When to use	Definition	Technical Term
اندازه می‌گیرد Term داخل همان Document چقدر مهم است.	Normalized frequency یک Term در یک Document.	TF
Term‌های رایج در همه Document‌ها را کم‌اهمیت می‌کند.	Logarithmic inverse document frequency یک Term در Collection.	IDF
یافتن Term‌های زیاد در یک Document ولی نادر در Collection.	Product دو مقدار TF و IDF.	TF-IDF
پیشگیری از تفسیر False Positive به عنوان Pattern واقعی.	در تعداد بسیار زیاد Test، Event تصادفی نادر نیز بارها رخ می‌دهد.	Bonferroni Principle
تبدیل Document به Set برای Near-duplicate detection.	Sequence متوالی از k Token در Document.	Shingle
Similarity بین Set‌ها و Binary asymmetric Data.	Ratio اندازه Intersection به Union دو Set.	Jaccard Similarity
تبدیل Set بزرگ به Signature کوتاه.	Randomized Hash که احتمال Collision آن برابر Jaccard است.	MinHash
کاهش Pair Comparison از حالت تقریباً Quadratic.	Hashing Family که Object‌های مشابه را با احتمال زیاد هم‌بافت می‌کند.	LSH
ساخت Candidate و Prune با استفاده از زیرمجموعه‌های Frequent.	Level-wise Algorithm برای Frequent Itemset Mining با Downward Closure.	Apriori
کاهش تعداد Candidate Pair‌ها در Pass 2.	بهبود Apriori برای Frequent Pair با Hash Bucket و Bitmap.	PCY
Geometry معمول و K-Means.	طول خط مستقیم بین دو Numeric Vector.	Euclidean Distance
Grid Movement و Robustness بیشتر نسبت به تغییرات مربعی.	Sum قدرمطلق Difference‌های Coordinate‌ها.	Manhattan Distance
Document و Sparse Vector؛ تمرکز بر Direction نه Magnitude.	Cosine زاویه بین دو Vector.	Cosine Similarity
Error Code و Binary Data.	تعداد Position‌های متفاوت در دو Sequence هم‌طول.	Hamming Distance
Spell Correction و Text/DNA Matching.	حداقل Cost برای تبدیل یک String به دیگری.	Edit Distance
جلوگیری از غلبه Attribute با Unit بزرگ.	تبدیل Scale Attribute‌ها به Range/Distribution قابل مقایسه.	Normalization
نمایش فشرده و Lossless از Support‌ها.	Frequent Itemset بدون Superset هم Support.	Closed Itemset
فشرده‌تر، ولی Support زیرمجموعه‌ها را نگه نمی‌دارد.	Frequent Itemset بدون هیچ Superset Frequent.	Maximal Itemset
کاهش Candidate با Relation‌های Tidset.	Algorithm عمودی برای Mining مستقیم Closed Itemset با Tidset.	CHARM

Clustering Algorithms

What it does / When to use	Definition	Technical Term
Numeric Data و Clusterهای تقریباً Spherical.	Partitioning Algorithm که هر Cluster را با Mean / Centroid نمایش می‌دهد.	K-Means
مقاوم‌تر به Outlier و مناسب Distance دلخواه.	Partitioning Algorithm با یک Data Point واقعی به عنوان Medoid.	K-Medoids
Dataset کوچک/متوسط با کیفیت بهتر از K-Means.	Search-based K-Medoids با Swap کردن Medoid و Non-medoid.	PAM
Sampling با Scaling K-Medoids.	اجرای PAM روی چند Sample از Dataset بزرگ.	CLARA
تعادل کیفیت و Cost بهتر از CLARA.	Randomized Search در Graph جواب‌های Medoid.	CLARANS
وقتی ساختار چندسطحی و Dendrogram لازم است.	ساخت Tree از Merge یا Split‌های پایینی Cluster.	Hierarchical Clustering
Data بسیار بزرگ و Clusterهای axis-aligned Gaussian.	Scalable K-Means-like Algorithm با Summary های N / SUM/SUMSQ.	BFR
Clusterهای Non-spherical و مقاومت بهتر به Outlier.	Hierarchical Algorithm با چند Representative Point کوچک شده.	CURE
Cluster با Shape دلخواه و Noise detection.	Density-based Algorithm با Eps و MinPts.	DBSCAN
Dataset با Density های متفاوت.	Ordering-based Density Algorithm با Reachability Distance.	OPTICS
Query سریع روی Spatial Data.	Hierarchical Grid Algorithm با Summary آماری Cell ها.	STING
کشف Cluster در بعضی Dimension ها.	Grid/Density Subspace Clustering برای High-dimensional Data.	CLIQUE
کشف Shape دلخواه و Noise با Model پیوسته Density.	Density-based Clustering مبتنی بر Influence Function و Density Attractor.	DENCLUE
نماینده Cluster در فضای Euclidean.	Mean Coordinate های Point های یک Cluster: لزوماً Data Point واقعی نیست.	Centroid
نماینده Cluster در فضای Non-Euclidean یا مقاوم‌تر به Outlier.	Data Point واقعی که Aggregate Distance آن تا بقیه کمینه است.	Clustroid / Medoid
Single, Complete, Average یا Centroid Merge.	Rule محاسبه Distance بین دو Cluster در Hierarchical Clustering.	Linkage
انتخاب Level یا تعداد Cluster با Cut کردن Tree.	Tree Diagram نمایش ترتیب Merge/Split های Hierarchical Clustering.	Dendrogram

[S1] Section 1 — Big Data, Data Mining and TF-IDF

Big Data

Definition: Dataset ی که Volume، Velocity یا Variety آن از توان Tool های معمول برای Processing در زمان معقول بیشتر است.

Purpose: برای مسئله‌هایی که به Distributed Storage و Parallel Processing نیاز دارند.

Execution: ابتدا Scale و نوع Data را تشخیص بده؛ سپس Framework مناسب مثل Hadoop را انتخاب کن.

Data Mining

Definition: فرایند کشف Pattern، Relation یا Knowledge مفید از Data.

Purpose: برای Description، Prediction، Classification، Clustering، Association و Outlier Detection.

Execution: Question را مشخص کن، Data را آماده کن، Algorithm را اجرا و Pattern را Evaluate کن.

TF (Term Frequency)

- Definition:** میزان تکرار Normalized یک Term در یک Document.
- Purpose:** نشان می‌دهد Term داخل همان Document چقدر برجسته است.
- Execution:** تعداد Term را بر بیشترین تکرار هر Term در همان Document تقسیم کن.

IDF (Inverse Document Frequency)

- Definition:** وزنی برای اندازه‌گیری نادر بودن Term در کل Document Collection.
- Purpose:** Term‌های بسیار عمومی را کم‌اهمیت و Term‌های کمیاب‌تر را پررنگ می‌کند.
- Execution:** نسبت N به تعداد Document‌های دارای Term را بگیر و $\log_2$ آن را حساب کن.

TF-IDF

- Definition:** Product مقدار TF و IDF برای یک Term در یک Document.
- Purpose:** یافتن Term‌ی که در یک Document زیاد ولی در کل Collection نادر است.
- Execution:** TF و IDF را جدا حساب کن و در هم ضرب کن.

Bonferroni Principle

- Definition:** وقتی تعداد Test‌ها بسیار زیاد است، Event کم‌احتمال صرفاً بر اثر Chance بارها رخ می‌دهد.
- Purpose:** برای جلوگیری از اشتباه‌گرفتن False Positive با Pattern واقعی در Massive Data.
- Execution:** تعداد Event‌های تصادفی مورد انتظار را از احتمال هر Event $\times$ تعداد Trial‌ها حساب و با یافته واقعی مقایسه کن.

Collaborative Filtering

- Definition:** Recommendation Method‌ی که از Rating/Behavior کاربران یا Item‌های مشابه استفاده می‌کند.
- Purpose:** پیشنهاد Product، Movie، یا Content بدون نیاز به Feature محتوایی کامل.
- Execution:** User-based: Similar User پیدا کن؛ Item-based: Similar Item پیدا کن؛ سپس Score ناشناخته را Estimate کن.

سه V

- Volume:** حجم خیلی زیاد
- Velocity:** تولید با سرعت بالا
- Variety:** شکل‌های مختلف داده
- Data Mining:** استخراج الگو/دانش مفید از داده.
- دو تکنیک:** خلاصه‌سازی (PageRank، Clustering) و Feature Extraction (Similarity، frequent itemsets و Recommender System).
- PageRank:** صفحه مهم است اگر از صفحات مهم لینک بگیرد.
- اصل Bonferroni:** در داده عظیم، رخداد خیلی نادر هم زیاد دیده می‌شود $\rightarrow$ False Positive.
- Hash:** $h(x) = x \bmod B$ | **Index:** بازبازی سریع رکورد | **دیسک:** خیلی کندتر از RAM؛ Algorithm باید I/O کم کند.

TF-IDF به زبان ساده

کلمه مفید = در تعداد کمی سند، زیاد تکرار شده. کلماتی مثل «و» یا the که همه جا هستند، مفید نیستند (stop words).

$$TF_{ij} = f_{ij} / \max_k(f_{kj})$$

$$IDF_i = \log_2(N / n_i)$$

$$TF-IDF = TF_{ij} \times IDF_i$$

مثال 1 – دقیقاً مثل سوال استاد (قدم به قدم)

فرض: تعداد کل اسناد $N = 1024$
کلمه w در 128 سند آمده ($n = 128$)
در سند z این کلمه 10 بار آمده
Frequentترین کلمه همان سند 100 بار آمده

$$\begin{aligned} \text{گام 1} \quad TF &= 10 / 100 = 0.1 \\ \text{گام 2} \quad IDF &= \log_2(1024 / 128) = \log_2(8) = 3 \\ &\text{چون } 8 = 3^2 \\ \text{گام 3} \quad TF-IDF &= 0.1 \times 3 = 0.3 \end{aligned}$$

پاسخ نهایی: 0.3

مثال 2 – برای فهم بهتر

اگر کلمه‌ای در همه اسناد باشد: $IDF = \log_2(1) = 0 \rightarrow$ بی‌اهمیت.
کلمه «هادوپ» فقط در 1 سند از 3 سند آمده، 8 بار؛ $\max$ همان سند = 20:

$$\begin{aligned} TF &= 8/20 = 0.4 \\ IDF &= \log_2(3/1) \approx 1.58 \\ TF-IDF &\approx 0.4 \times 1.58 = 0.63 \end{aligned}$$

پاسخ نهایی: ≈ 0.63

Collaborative Filtering

- **User-based**: کاربران شبیه تو چه چیزهایی را پسندیده‌اند؟ همان‌ها را به تو پیشنهاد بده.
 - **Item-based**: آیتم‌های شبیه آنچه پسندیده‌ای را پیشنهاد بده.
- در درس به Recommendation Engine (مثل Netflix و Amazon) و Behavioral Data اشاره شده.

[S2] Section 2 – Cluster Computing, RDBMS and Hadoop

RDBMS

Definition: Database Management System مبتنی بر Relation/Table، Schema و Transaction های ACID.
Purpose: برای Structured Data، Query پیچیده و Transaction دقیق.
Execution: Data را در Table ذخیره کن و با SQL و Key/Constraint مدیریت کن.

Cluster Computing

Definition: اجرای یک Workload روی چند Node متصل که به صورت یک System همکاری می‌کنند.
Purpose: برای Fault Tolerance، Horizontal Scaling، Parallel Processing، High Availability.
Execution: Data و Task را بین Node ها Partition کن، Load را Balance و Failure را Recover کن.

Hadoop

Definition: Framework متن‌باز برای Distributed Storage و Batch Processing روی Commodity Hardware.
Purpose: برای File های بسیار بزرگ و Job های Sequential/Batch با Fault Tolerance.
Execution: Data را در HDFS ذخیره و Computation را با MapReduce نزدیک Data اجرا کن.

HDFS

Definition: Distributed File System که File را به Block تقسیم و روی چند DataNode Replicate می‌کند.
Purpose: Storage مقیاس‌پذیر و مقاوم به Failure برای File های بزرگ.
Execution: NameNode Metadata را نگه می‌دارد؛ DataNode ها Block ها را ذخیره می‌کنند.

محدودیت‌های RDBMS: Scalability گران، سرعت، Fault Tolerance، پردازش پیشرفته روی ترابایت/پتابایت. ویژگی‌های **Cluster (حفظ کن):** High Availability, Fault Tolerance, Data Replication, Load Balancing, Automated Failover, Backup & Recovery, Monitoring, Scalability.

موضوع	RDBMS سنتی	Cluster / Hadoop
سخت‌افزار	سرور گران	commodity ارزان
مقیاس	عمودی (تقویت یک سرور)	افقی (افزودن نود)
Fault Tolerance	غالباً سخت‌افزاری	نرم‌افزاری با replication
ایده پردازش	داده را پیش‌کد ببر	کد را پیش‌داده ببر

نمونه پاسخ آماده برای امتحان

- (۱) RDBMS برای داده ساخت‌یافته و Transaction ACID عالی است، ولی وقتی داده خیلی بزرگ شود هزینه و سرعت مشکل می‌شود.
- (۲) Cluster Computing با سرورهای ارزان، کپی داده و failover خودکار، مقیاس افقی می‌دهد.
- (۳) جمله طلایی: در Hadoop به‌جای کشیدن همه داده به یک ماشین، برنامه را به جایی می‌فرستیم که داده هست.

Hadoop: Scalable + Fault Tolerant | HDFS + MapReduce | NameNode / DataNode / JobTracker / TaskTracker | معمولاً هر chunk سه کپی دارد.
SQL در برابر NoSQL: SQL اسکیمای ثابت و ACID؛ NoSQL اسکیمای انعطاف‌پذیر و مقیاس افقی.

تکمله A – حفظیات KDD، Data Mining، مخازن داده و Hadoop [D1]

Data Mining در برابر KDD

Data Mining استخراج دانش از حجم زیاد داده است؛ نام‌های نزدیک آن: استخراج دانش، تحلیل داده/الگو، باستان‌شناسی داده، لایروبی داده و KDD. باین‌حال در تعریف فرایندی، Data Mining فقط یک مرحله از KDD است.

1. **Data Cleaning:** حذف Noise، تناقض و داده غلط
2. **Data Integration:** ترکیب چند منبع
3. **Data Selection:** Data Selection مرتبط با مسئله
4. **Data Transformation:** تبدیل، Aggregation و Normalization
5. **Data Mining:** اجرای روش استخراج الگو
6. **Pattern Evaluation:** تشخیص الگوهای معتبر و جالب
7. **Knowledge Presentation:** نمایش و مصورسازی دانش

KDD — End-to-end Example

- مسئله: پیش‌بینی Customer Churn یک اپراتور.
- Data Cleaning: حذف Record خراب و تکمیل Missing Value.
 - Data Integration: ترکیب Call Log، Billing و Support Ticket.
 - Data Selection: انتخاب Customerهای فعال و Attributeهای مرتبط.
 - Data Transformation: ساخت Feature مثل میانگین مصرف و تعداد Complaint.
 - Data Mining: اجرای Classification Model.
 - Pattern Evaluation: بررسی Precision/Recall و False Positive.
 - Knowledge Presentation: Dashboard فهرست Customerهای پرریسک.

وظایف Data Mining: حفظیات مهم

وظیفه	تعریف	مثال
Descriptive	توصیف ساختار و روابط موجود	Clustering, Association Rules
Predictive	پیش‌بینی کلاس یا مقدار ناشناخته	Classification, رگرسیون
Characterization	خلاصه ویژگی‌های یک کلاس هدف	ویژگی مشتریان وفادار
Discrimination	مقایسه کلاس هدف با کلاس متضاد	وفادار در برابر ریزشی
Outlier Analysis	کشف شیء ناسازگار با الگوی عمومی	تقلب کارت اعتباری
Evolution Analysis	مدل‌کردن روند تغییر در زمان	Time Series و الگوی دوره‌ای

Classification: Supervised Learning؛ مدل می‌تواند IF-THEN، درخت تصمیم، فرمول یا شبکه عصبی باشد. روش‌های نام‌برده در لکچر: Naive Bayes، SVM و KNN.

Clustering: بدون برچسب؛ گروه‌های طبیعی را کشف می‌کند. هر Cluster بعداً می‌تواند یک کلاس تلقی شود.

Outlier: ممکن است Noise باشد، اما گاهی مهم‌ترین دانش داده است؛ نباید همیشه حذف شود.

انواع Frequent Pattern

- **Frequent Itemset:** اقلامی که مکرراً باهم رخ می‌دهند؛ مثل شیر و نان
- **Frequent Subsequence:** دنباله عملیاتی که مکرراً با همان ترتیب رخ می‌دهد
- **Frequent Substructure:** مثل گراف یا درخت

چه الگویی «جالب» است؟

قابل فهم، معتبر، بالقوه مفید، جدید و نمایش‌دهنده دانش باشد. معیارهای عینی مانند Support و Confidence آستانه‌ای دارند که کاربر تعیین می‌کند.

معماری سیستم Data Mining

1. پایگاه Data Warehouse/Web یا مخزن دیگر
2. سرور پایگاه یا Data Warehouse
3. پایگاه دانش (Knowledge Base)
4. موتور Data Mining
5. مازول Pattern Evaluation
6. واسط کاربر

انواع Data Repository

مخزن	تعریف و نکته
Relational DB	جدول، سطر/تاپل، ستون/صفت و کلید یکتا
Data Warehouse	داده یکپارچه چند منبع؛ پاک‌سازی، تبدیل و بارگذاری؛ مدل چندبعدی
Data Cube	نمای چندبعدی؛ نمونه ابعاد: زمان، کالا، مکان
Transactional DB	هر رکورد یک TID و فهرست اقلام دارد
Temporal	دارای صفت زمانی
Sequence	رویدادهای مرتب، با یا بدون timestamp
Time-series	اندازه‌گیری تکراری یک مقدار در زمان
Spatial	GIS/CAD/تصویر؛ نمایش Raster یا Vector
Text / Multimedia	متن، تصویر، صدا و ویدئو
Heterogeneous / Legacy	ترکیب سامانه‌های مستقل، قدیمی و ناهمگون
Data Stream	بسیار بزرگ/نامتناهی، پویا، یک یا چند scan، پاسخ سریع و بلادرنگ

چالش‌های Data Mining

- استخراج تعاملی در سطوح مختلف انتزاع و استفاده از دانش پس‌زمینه
- زبان Query ی Data Mining، نمایش و مصورسازی نتایج

- داده ناقص، Noise و تشخیص الگوهای باکیفیت
 - کارایی و Scalability؛ Algorithm موازی، توزیع شده و افزایشی
 - انواع داده پیچیده و پایگاه های ناهمگون
- سطوح اتصال Mining به DB/DW (فهرست حفظی): No Coupling, Loose Coupling, Semitight Coupling و Tight Coupling.

Bonferroni – مثال حفظی هتل

$$P(\text{one pair meets in one day}) = (0.01 \times 0.01) / 10^5 = 10^{-9}$$

$$E(\text{random double meetings}) \approx C(10^9, 2) \times C(1000, 2) \times 10^{-18} \approx 250,000$$

نتیجه: حتی با داده کاملاً تصادفی، حدود ۲۵۰ هزار جفت «مشکوک» پیدا می شود؛ پس در داده عظیم، نادر بودن به تنهایی دلیل معناداری نیست.

Hash، Index و دیسک – جزئیات

- تابع هش خوب کلیدها را تقریباً یکنواخت بین B باکت توزیع می کند.
- برای $h(x) = x \bmod B$ معمولاً B اول انتخاب می شود تا الگوهای ورودی توزیع را خراب نکنند.
- برخورد هش یعنی چند کلید در یک باکت؛ پس بعد از یافتن باکت باید مقدار واقعی بررسی شود.
- دیسک بلوکی است؛ بلوک کوچک ترین واحد انتقال دیسک $\leftrightarrow$ RAM است.
- خواندن بلوک حدود 10ms و تقریباً 10^5 برابر خواندن کلمه از RAM کندتر است.

Hadoop – جزئیات حفظی

- فلسفه: commodity server ارزان + Fault Tolerance نرم افزاری + انتقال کد به داده
 - اکوسیستم: HDFS، MapReduce، HBase، Hive و Pig
 - NameNode: metadata و محل بلوک ها؛ DataNode: خود بلوک ها
 - JobTracker: زمان بندی Job؛ TaskTracker: اجرای Map/Reduce (معماری Hadoop 1)
 - Standalone: فایل محلی و یک JVM
 - Pseudo-distributed: یک ماشین ولی daemon ها HDFS جدا
 - Fully distributed: چند نود و replication
 - Cloud: AWS/S3 و Azure HDInsight/Blob Storage
 - نامناسب: پایگاه Transaction ی با update بسیار مکرر
- کاربردهای تجاری: Ad Targeting و Risk Modeling، Customer Churn، Recommendation Engine.
- JVM: Java به bytecode کامپایل می شود؛ هر process جاوا JVM و حافظه مجزا دارد. فایل class مستقل از سیستم عامل است ولی JVM پلتفرم وابسته است.

[S3] Section 3 – MapReduce

MapReduce

Definition: Distributed Programming Model برای تبدیل Input به Key-Value Pair و Aggregate کردن Value های هم Key.

Purpose: برای Batch Processing موازی روی Data حجیم با Fault Recovery خودکار.

Execution: (۱) Map (بنویس ۲) System، Shuffle/Sort/Partition را انجام می دهد (۳) Reduce بنویس.

Map

Definition: تابعی که هر Input Record را به صفر، یک یا چند (key, value) تبدیل می کند.

Purpose: استخراج Key و Intermediate Value به صورت Parallel.

Execution: هر Mapper یک Chunk مستقل را می خواند و emit می کند.

Shuffle

Definition: مرحله System برای انتقال، Sort و Group کردن همه Value های دارای Key یکسان.

Purpose: تضمین می‌کند Reduce همه Value های یک Key را یک جا ببیند.

Execution: Partitioner مقصد را تعیین می‌کند و System داده‌های میانی را روی Network جابه‌جا می‌کند.

Reduce

Definition: تابعی که یک Key و List همه Value های آن را به Output نهایی تبدیل می‌کند.

Purpose: Aggregation, Join, Count, Sum یا ساخت نتیجه نهایی.

Execution: برای هر Key دقیقاً روی Group متناظر اجرا می‌شود.

Combiner

Definition: Optional Local Reduce در سمت Mapper.

Purpose: کاهش حجم Intermediate Data روی Network.

Execution: فقط وقتی Commutative و Associative، Operation است اجرا کن؛ جای Reduce نهایی نیست.

تو فقط دو تابع می‌نویسی؛ سیستم بقیه کار را می‌کند.

1. **Map:** هر تکه ورودی را بخوان و جفت‌های (کلید، مقدار) بساز.
2. **Shuffle:** همه مقدارهایی که کلید یکسان دارند را در یک لیست بگذار.
3. **Reduce:** برای هر کلید، لیست مقدارها را ترکیب کن (جمع، شمارش، ...).

Map: input → (key, value)

Shuffle: (k,v1), (k,v2), ... → (k, [v1, v2, ...])

Reduce: (k, [v1, v2, ...]) → (k, result)

مثال خیلی ساده – شمارش کلمه

دو جمله داریم: «big data» و «big data systems»

Map از جمله 1: (big,1) (data,1)
Map از جمله 2: (big,1) (data,1) (systems,1)

بعد از Shuffle:
big → [1, 1]
data → [1, 1]
systems → [1]

Reduce (جمع):
big = 2
data = 2
systems = 1

پاسخ نهایی: big=2 , data=2 , systems=1

Combiner: اگر عمل Reduce جابجایی‌پذیر و انجمنی باشد (مثل جمع)، می‌توان قبل از شبکه یک Reduce محلی زد تا ترافیک کم شود.

اگر این خراب شود	چه کار می‌کنند؟
Master	معمولاً کل Job از نو شروع می‌شود
Map Worker	همان Map ها روی نود سالم دوباره اجرا می‌شوند
Reduce Worker	Reduce دوباره اجرا می‌شود

Workflow Systems: به جای فقط Map→Reduce، یک گراف DAG از توابع (Clustera, Hyracks).
Pregel: سیستم محاسبات گرافی با SuperStep و Checkpoint برای Fault Tolerance در Algorithms بازگشتی.
PageRank: تکراری:

$$v^{(t+1)} = M \times v^{(t)}$$

[S4] Section 4 — Relational Algebra with MapReduce

Relational Algebra

Definition: مجموعه Operation های رسمی برای تبدیل Relation ها به Relation جدید.
Purpose: پایه مفهومی Query Processing شامل Selection, Projection, Union, Difference و Join.
Execution: هر Operation را به Map برای تولید Key و Reduce برای ترکیب Record های مرتبط تبدیل کن.

عملیات	ایده ساده
Selection	فقط تاپلهایی که شرط دارند را نگه دار
Projection	بعضی ستون ها را Vector؛ تکراری ها را در Reduce حذف کن
Union	همه را بفرست؛ تکراری یکی شود
Intersection	فقط چیزی که در هر دو آمده
Difference R-S	چیزی که فقط در R است
Join	کلید = ستون مشترک؛ دو طرف را به هم بچسبان
Group + Aggregation	کلید = گروه؛ Reduce = SUM/COUNT/AVG

مثال Join برای مبتدی

جدول R: (علی، تهران) و (مینا، اصفهان)
 جدول S: (تهران، ایران) و (اصفهان، ایران)
 می خواهیم روی «شهر» Join کنیم:

Map: کلید = شهر
 تهران → از R: علی و از S: ایران
 اصفهان → از R: مینا و از S: ایران

Reduce تهران: (علی، تهران، ایران)
 Reduce اصفهان: (مینا، اصفهان، ایران)

پاسخ نهایی: (علی، تهران، ایران) و (مینا، اصفهان، ایران)

Multi-way Join: Join همزمان چند جدول. هزینه انتقال داده مهم است؛ Replication Rate نشان می دهد هر ورودی چند بار کپی می شود.

[S5] Section 5 — Matrix Processing with MapReduce

Matrix-Vector Multiplication

Definition: Operation ی که Dot Product هر Row Matrix M با Vector V را در u قرار می دهد.
Purpose: هسته بسیاری از Graph/ML Algorithm ها مانند PageRank.
Execution: Mapper مقدار $m_{ij} \times v_j$ را با Key=i می سازد و Reducer تمام Partial Product های i را Sum می کند.

Matrix-Matrix Multiplication

Definition: Operation ی که p_{ik} را از Dot Product سطر i Matrix اول و ستون k Matrix دوم می‌سازد.

Purpose: اجرای Linear Algebra بزرگ‌تر از RAM.

Execution: Job اول روی Index مشترک Partial Product، z می‌سازد؛ Job دوم روی $Key=(i,k)$ آن‌ها را Sum می‌کند.

هدف $Matrix \times Vector$: برای هر سطر Matrix یک عدد بساز = ضرب داخلی آن سطر در Vector.

$$u_i = m_{i1} \cdot v_1 + m_{i2} \cdot v_2 + \dots + m_{in} \cdot v_n$$

$$\text{Map: } (i, m_{ij} \times v_j)$$

$$\text{Reduce: } u_i = \sum_j (m_{ij} \times v_j)$$

مثال کامل – انگار اولین بار می‌بینی

Matrix و Vector:

$$M = \begin{bmatrix} 1 & 0 & 5 \\ 7 & 3 & 0 \\ 1 & 3 & 5 \end{bmatrix} \quad V = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

گام Map: هر خانه Matrix را در عنصر متناظر Vector ضرب کن. کلید = شماره سطر.

سطر 1: $(2=2 \times 1, 1)$, $(0=4 \times 0, 1)$, $(30=6 \times 5, 1)$

سطر 2: $(14=2 \times 7, 2)$, $(12=4 \times 3, 2)$, $(0=6 \times 0, 2)$

سطر 3: $(2=2 \times 1, 3)$, $(12=4 \times 3, 3)$, $(30=6 \times 5, 3)$

گام Shuffle: مقدارهای هر سطر را کنار هم بگذار.

کلید 1 $\rightarrow [30, 0, 2]$

کلید 2 $\rightarrow [0, 12, 14]$

کلید 3 $\rightarrow [30, 12, 2]$

گام Reduce: جمع کن.

$$u_1 = 2 + 0 + 30 = 32$$

$$u_2 = 14 + 12 + 0 = 26$$

$$u_3 = 2 + 12 + 30 = 44$$

پاسخ نهایی: $U = [32, 26, 44]$

$Matrix \times Matrix$ (دو Job)

$$p_{ik} = \sum_j (m_{ij} \times n_{jk})$$

Job1: روی z جوین کن و ضرب‌های جزئی بساز. Job2: برای هر (i,k) جمع بزن.

چک یک خانه

خانه $(1,1)$ از $P: 1 \times 2 + 0 \times 4 + 5 \times 6 = 2 + 0 + 30 = 32$

تکمله B – اجرای داخلی و توسعه‌های MapReduce [D2]

Grouping و Map Task, Partitioning

- عنصر ورودی (تاپل/سند) نباید بین دو chunk نصف شود.
- Map می‌تواند برای یک ورودی صفر، یک یا چند زوج و حتی چند کلید یکسان تولید کند.

- اگر Reducer r باشد، $h(k)$ یکی از 0 تا $r-1$ را انتخاب می‌کند.
- هر Map برای هر Reduce یک فایل محلی میانی دارد؛ بنابراین r فایل می‌سازد.
- شرط اصلی: کلیدهای یکسان حتماً به یک Reduce بروند؛ کلیدهای متفاوت می‌توانند هم‌مقصد شوند.

$$k_1 = k_2 \Rightarrow h(k_1) = h(k_2)$$

$$(k, v_1), (k, v_2), \dots \Rightarrow (k, [v_1, v_2, \dots])$$

Grouping سیستمی را با GROUP BY رابطه‌ای اشتباه نکن: Grouping فقط مقادیر هم‌کلید را کنار هم قرار می‌دهد؛ Aggregation عملی است که کاربر در Reduce می‌نویسد.

Combiner – دام مهم امتحانی

$$(a \circ b) \circ c = a \circ (b \circ c) \text{ (Associative)}$$

$$a \circ b = b \circ a \text{ (Commutative)}$$

Combiner جای Reduce نهایی را نمی‌گیرد؛ فقط تکراری‌ها/مقادیر همان Map را ترکیب می‌کند. مثلاً در Projection تکراری‌های دو Map مختلف فقط در Reduce نهایی حذف می‌شوند.

عملیات رابطه‌ای – Algorithm دقیق

عملیات	Map	Reduce
Selection σ	اگر $C(t)$: emit(t, t)	عبور؛ اغلب Map-only
Projection π	emit(t', t')	یک t' برای حذف تکرار
Union	از R و S : emit(t, t)	یک بار t
Intersection	R : emit(t, R), S : emit(t, S)	اگر هر دو برچسب بودند
Difference $R-S$	همراه برچسب منبع	فقط labels= $\{R\}$
Join	کلید=همه صفات مشترک	حاصل Cartesian Product محلی دو طرف
Grouping/Agg	کلید=صفات گروه	SUM/COUNT/AVG/MIN/MAX

تفاضل جابجایی‌پذیر نیست: $R-S$ با $S-R$ فرق دارد. در Intersection حتماً نام رابطه را بفرست؛ دو تکرار از یک رابطه به معنی حضور در هر دو رابطه نیست.

Join: اگر برای یک کلید، ۲ تاپل R و ۳ تاپل S باشد، Reduce باید $6=3 \times 2$ خروجی بسازد.

Matrix×Vector وقتی Vector در RAM جا نمی‌شود

Matrix را به نوارهای عمودی و Vector را به قطعات افقی متناظر تقسیم کن؛ تعداد نوارها طوری باشد که قطعه Vector و chunk Matrix در حافظه یک نود جا شود. هر Map سهم جزئی u_i را می‌سازد و Reduce سهم نوارهای مختلف را روی i جمع می‌کند.

Matrix×Matrix: دو Job در برابر یک Job

دو Job: Job1 روی j جوین و $(m_{ij}n_{jk}, (i, k))$ تولید می‌کند؛ Job2 روی (i, k) جمع می‌زند.

یک Job: هر m_{ij} برای تمام k ها و هر n_{jk} برای تمام i ها تکثیر می‌شود. Job کمتر، ولی Replication و ترافیک بیشتر.

Transitive Closure (Transitive Closure)

$$P = E \cup \{ (x, y) \mid \exists z : P(x, z) \wedge P(z, y) \}$$

اگر مسیر $(1, 2)$ و $(2, 3)$ داشته باشیم، Join روی گره میانی 2 مسیر $(1, 3)$ را می‌سازد. دو نوع Task: Duplicate و Elimination Task. مسیر جدید پس از Duplicate Elimination به Join Taskهای مربوط به دو سر مسیر فرستاده می‌شود؛ توقف وقتی مسیر جدیدی ساخته نشود.

- اجرای همگام در مرحله‌های SuperStep
- Checkpoint = کپی وضعیت همه Taskها
- در خرابی، بازگشت به آخرین Checkpoint
- زمان ترمیم باید خیلی کمتر از میانگین زمان بین دو خرابی باشد

Replication Rate و Communication Cost, Reducer Size

$$\text{Communication Cost} = \sum_{\text{task}} |\text{Input}_{\text{task}}|$$

$$\rho = (\# \text{ Map outputs}) / (\# \text{ input elements})$$

Reducer Size (q): بیشترین تعداد مقدار مجاز برای یک q. Reduce کوچک‌تر → Reduce‌های بیشتر و موازی‌سازی بیشتر؛ اگر ورودی Reduce در RAM جا شود I/O کمتر می‌شود.

Trade-off: یک Reduce هزینه انتقال حداقل ولی زمان اجرا زیاد؛ Reduce‌های بیشتر زمان کمتر ولی احتمال تکثیر/هزینه بیشتر.

Multi-way Join یک مرحله‌ای

برای $R(A,B) \bowtie S(B,C) \bowtie T(C,D)$ ، تعداد باکت B برابر b و C برابر c است؛ پس $k=bc$ Reducer

- $S(v,w)$ فقط به Reduce با آدرس $(h(v), g(w))$ می‌رود.
- $R(u,v)$ چون C ندارد به تمام c ستون تکثیر می‌شود.
- $T(w,x)$ چون B ندارد به تمام b سطر تکثیر می‌شود.

$$CC_{\text{reduce}} = s + c \cdot r + b \cdot t \text{ subject to } b \cdot c = k$$

$$b^* = \sqrt{(k \cdot r / t)} \quad c^* = \sqrt{(k \cdot t / r)}$$

$$CC_{\text{reduce}}(\min) = s + 2\sqrt{(krt)}$$

$$CC_{\text{total}} = r + 2s + t + 2\sqrt{(krt)}$$

مثال Multi-way Join

اگر $|R|=|T|=100$ و $k=4$ ، انتخاب بهینه $b=c=2$ است.

$$nCC_{\text{reduce}} = s + 2 \times 100 + 2 \times 100 = s + 400 \quad \text{هر } S \text{ یک بار} + \text{هر } T \text{ دو بار} + \text{هر } R \text{ دو بار}$$

روش دو مرحله‌ای: اگر احتمال تطابق R و S برابر p باشد، اندازه داده میانی تقریباً prs و هزینه $O(r+s+t+prs)$ است. انتخاب روش به اندازه روابط و خروجی میانی بستگی دارد.

[S6] Section 6 — Shingle, Jaccard, MinHash, LSH and PCY

Shingle

Definition: Sequence متوالی از k Token در یک Document.

Purpose: تبدیل Document به Setی که بخشی از ترتیب Local را حفظ می‌کند.

Execution: Window طول k را روی متن Slide کن و برای Duplicate، Setها را حذف کن.

Jaccard Similarity

Definition: Ratio اندازه Intersection به Union دو Set.

Purpose: مقایسه Set، Basket، Shingle و Binary asymmetric Data.

Execution: تعداد عضوهای مشترک را بر تعداد عضوهای موجود در حداقل یک Set تقسیم کن.

MinHash

Definition: Randomized Hashing که Collision Probability آن برای دو Set برابر Jaccard است.

Purpose: ساخت Signature کوتاه با حفظ تقریبی Similarity.

Execution: چند Permutation/Hash Function بساز و برای هرکدام Minimum Row دارای 1 را ثبت کن.

LSH (Locality-Sensitive Hashing)

Definition: Hashing Scheme ی که Object های مشابه را با احتمال بیشتر در یک Bucket قرار می دهد.

Purpose: کاهش Pair Comparison و تولید Candidate Pair.

Execution: Signature را به b Band با r Row تقسیم کن؛ Match کامل در حداقل یک Band کافی است.

PCY

Definition: Two-pass Algorithm برای Frequent Pair Mining با Hash Bucket و Bitmap.

Purpose: Prune کردن Candidate Pair بیشتر از Apriori ساده.

Execution: Pass1 Item و Bucket را Count کن؛ Pass2 فقط Pair های با Item Frequent و Bucket Frequent را Count کن.

**Document → Shingles → MinHash Signature → LSH →
Candidates → Exact check**

Shingle چیست؟

یک پنجره به طول k روی متن می گذاری و هر بار یک تکه برمی داری.

مثال

متن: abcab و k = 2

تکه ها: ab , bc , ca , ab

مجموعه بدون تکرار: {ab, bc, ca}

Jaccard

$$J(A,B) = |A \cap B| / |A \cup B|$$

مثال با میوه

A = {سیب، پرتقال، موز} و B = {موز، انگور}

اشتراک = {موز} → 1

اجتماع = {سیب، پرتقال، موز، انگور} → 4

J = 1/4 = 0.25

Distance = 1 - 0.25 = 0.75

پاسخ نهایی: J = 0.25

MinHash

عناصر را تصادفی مرتب کن. برای هر مجموعه، اولین عنصری که در آن ترتیب می بینی را به عنوان hash Vector.

$$\Pr[h\pi(C_1) = h\pi(C_2)] = J(C_1, C_2)$$

مثال

$$B = \{2,3,4,5\} \text{ و } A = \{1,3,4\}$$

$$0.4 = 2/5 = \text{Jaccard واقعی}$$

یک ترتیب تصادفی فرضی: 3, 1, 5, 2, 4

اولین عنصر A در این ترتیب = 3

اولین عنصر B در این ترتیب = 3

پس hashها برابر شدند

اگر خیلی ترتیب تصادفی بگیری، نسبت دفعاتی که برابر می‌شوند حدود 0.4 می‌شود.

LSH

امضا را به چند نوار (band) تقسیم کن. اگر دو سند حتی در یک نوار کاملاً یکسان بودند، آن‌ها را candidate حساب کن و بعد دقیق چک کن.

$$P = 1 - (1 - s^r)^b$$

که در آن:

s = Similarity واقعی

r = تعداد سطر در هر band

b = تعداد bandها

مثال عددی

$$b = 20, r = 5, s = 0.8$$

$$s^r = 0.8^5 \approx 0.328$$

$$(1 - 0.328)^{20} = 0.672^{20} \approx 0.00035$$

$$P = 1 - 0.00035 \approx 0.99965$$

یعنی حدود 99.97٪ احتمال دارد جفت واقعاً مشابه را پیدا کنیم.

پاسخ نهایی: $P \approx 0.99965$

PCY: در گذر اول علاوه بر شمارش ارقام، جفت‌ها را به bucket هش کن. در گذر دوم فقط جفتی را بشمار که هر دو قلم frequent باشند و bucketشان هم frequent باشد.

[S7] Section 7 — Distance, Similarity and Tuple Comparison

Metric

Definition: Distance Function ی که Non-negativity, Identity, Symmetry و Triangle Inequality را ارضا می‌کند.

Purpose: اندازه‌گیری معتبر جدایی دو Object و استفاده در Search یا Clustering.

Execution: نوع Attribute و Geometry مسئله را تشخیص بده، سپس Metric مناسب را انتخاب و در صورت نیاز Normalize کن.

Euclidean Distance (L_2)

Definition: طول خط مستقیم بین دو Numeric Vector.

Purpose: فضای Continuous و Clusterهای تقریباً Spherical.

Execution: Difference هر Dimension را Square، Sum و Square Root کن.

Manhattan Distance (L_1)

Definition: Sum قدر مطلق Difference ها در همه Dimension ها.
Purpose: حرکت Grid-like و حالتی که Difference خطی مهم است.
Execution: قدر مطلق Difference هر Dimension را جمع کن.

Cosine Similarity

Definition: Cosine زاویه بین دو Vector؛ Magnitude را تا حد زیادی نادیده می‌گیرد.
Purpose: Text Vector های Sparse و مقایسه Direction دو Profile.
Execution: Dot Product را بر Product دو Vector Norm تقسیم کن.

Hamming Distance

Definition: تعداد Position های متفاوت دو Sequence هم‌طول.
Purpose: Binary Code، String، Error Detection.
Execution: Position ها را یک‌به‌یک Compare و Mismatch ها را Count کن.

Edit Distance

Definition: کمترین Cost Operation های لازم برای تبدیل یک String به String دیگر.
Purpose: String Matching، Spell Checking، DNA.
Execution: با Dynamic Programming حداقل Insert/Delete/Replace را محاسبه کن؛ فرمول LCS جزوه فقط حالت Insert/Delete است.

چهار شرط Metric: نامنفی، صفر فقط وقتی دو شیء یکی باشند، تقارن، نامساوی مثلثی.

$$\text{Euclidean } L_2 = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots}$$

$$\text{Manhattan } L_1 = |x_1 - y_1| + |x_2 - y_2| + \dots$$

$$\text{Chebyshev } L_\infty = \max |x_i - y_i|$$

مثال روی یک جفت نقطه

$X = (1, 2)$ و $Y = (4, 6)$

$$L_2 = \sqrt{(1-4)^2 + (2-6)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

$$L_1 = |1-4| + |2-6| = 3 + 4 = 7$$

$$L_\infty = \max(3, 4) = 4$$

$$\text{Nominal Dissimilarity: } d = (p - m) / p$$

که در آن:

p = تعداد کل صفات

m = تعداد صفاتی که مقدارشان یکی است

مثال Nominal

شخص 1: (مرد، تهران، لیسانس)

شخص 2: (زن، تهران، لیسانس)

$$p = 3$$

$$m = 2 \text{ (شهر و مدرک یکسان‌اند)}$$

$$d = (3-2)/3 = 1/3 \approx 0.33$$

پاسخ نهایی: 1/3

Binary: $q = \text{هر دو } 1 \mid t \mid s = (1,0) \mid r = (0,1) = \text{هر دو } 0$

Symmetric Binary: $d = (r + s) / (q + r + s + t)$

Asymmetric Binary: $d = (r + s) / (q + r + s)$

Jaccard Binary: $J = q / (q + r + s)$

Cosine برای متن خوب است. **Hamming** = تعداد خانه‌های متفاوت. **Edit distance** = $|X| + |Y| - 2 \times |LCS|$. **Data Matrix**: سطر = تاپل، ستون = صفت. **Dissimilarity Matrix**: Distance بین تاپل‌ها؛ Diameter اصلی صفر.

[S8] Section 8 — Apriori, Closed Itemset, Maximal Itemset and CHARM

Support

Definition: Fraction Transactions: ی که یک Itemset یا Rule را پوشش می‌دهند.

Purpose: اندازه‌گیری فراگیری Pattern در کل Database.

Execution: Support Count را بر تعداد کل Transaction‌ها تقسیم کن.

Confidence

Definition: Conditional Probability رخداد Consequent وقتی Antecedent رخ داده است.

Purpose: اندازه‌گیری Reliability یک Association Rule.

Execution: $\text{Support}(A \cup B)$ را بر $\text{Support}(A)$ تقسیم کن.

Apriori

Definition: Level-wise Frequent Itemset Algorithm مبتنی بر Downward-closure Property.

Purpose: کشف Itemset‌های Frequent و ساخت Association Rule.

Execution: از L_1 شروع کن؛ C_k Candidate را Join، با زیرمجموعه‌های Infrequent Prune و Support را Count کن.

Closed Itemset

Definition: Frequent Itemset ی که هم Superset ندارد.

Purpose: فشردگی Lossless Frequent Pattern‌ها همراه با حفظ Support.

Execution: Superset‌ها را بررسی کن یا Closure را با Tidset به دست آور.

Maximal Itemset

Definition: Frequent Itemset ی که هیچ Superset ندارد.

Purpose: فشردگی قوی‌تر مرز Frequent‌ها، بدون حفظ Support دقیق زیرمجموعه‌ها.

Execution: بررسی کن هیچ Extension آن $\text{Support} \geq \text{minsup}$ نداشته باشد.

Definition: Tidset Intersection بر مبنی Vertical Closed-itemset Mining Algorithm.

Purpose: استخراج مستقیم Closed Itemset بدون تولید همه Frequent Itemset ها.

Execution: Itemset های هم Prefix را ترکیب، Tidset ها را Intersect و با Equality/Subset Relation هرس کن.

$$\text{support}(X) = \text{support_count}(X) / |D|$$

$$\text{support}(A \Rightarrow B) = \text{support}(A \cup B)$$

$$\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

$$M \subseteq C \subseteq F$$

مثال Association Rule

از 100 سبد خرید: نان در 40 سبد، نان+شیر در 30 سبد.

قانون: نان $\Rightarrow$ شیر

$$\text{support} = 30/100 = 0.3$$

$$\text{confidence} = 30/40 = 0.75 = 75\%$$

یعنی 75% کسانی که نان گرفته‌اند شیر هم گرفته‌اند.

پاسخ نهایی: conf = 75%

نوع	معنی ساده
Closed	اگر قلمی اضافه کنی، support دیگر همان عدد قبلی نمی‌ماند
Maximal	اگر قلمی اضافه کنی، دیگر frequent نیست

مثال Maximal و Closed

Transactions: {A,B,C} , {A,B} , {A,B,C} , {A,C} و حداقل شمارش = 2

$$A=4, B=3, C=3$$

$$AB=3, AC=3, BC=2$$

$$ABC=2$$

Closed ها: A , AB , AC , ABC

(B closed نیست چون AB همان support=3 را دارد)

Maximal فقط: ABC

اصل Apriori: اگر مجموعه‌ای frequent باشد، همه زیرمجموعه‌هایش هم frequent اند. پس کاندید بزرگ را فقط از frequent های کوچک می‌سازیم.

CHARM: Algorithm استخراج Closed با اشتراک tidset ها.

تکمله C — جزئیات CHARM و Similarity, MinHash, LSH, PCY [D3]

Characteristic Matrix

• هر سطر = یک shingle یکتا؛ هر ستون = یک سند

• خانه 1 یعنی shingle در سند وجود دارد؛ Matrix معمولاً بسیار sparse است.

• AND دو ستون = اشتراک؛ OR = اجتماع.

Set در برابر Bag: Set تکرار shingle را حذف می‌کند؛ Bag فراوانی را نگه می‌دارد. MinHash کلاسیک روی Set است.

انتخاب k و Hash کردن Shingle

- k خیلی کوچک → اسناد نامرتبط هم اشتراک زیاد دارند.
- در لکچر: k≈5 برای سند کوتاه و k≈10 برای سند بلند.
- می‌توان shingle را به شناسه ۴ بایتی hash کرد تا فضا کم شود.
- Collision می‌تواند Similarity را کمی بیش‌برآورد کند؛ فضای hash باید بزرگ باشد.

اثبات MinHash – جملهای که حفظ می‌کنی

در اجتماع $C_1 \cup C_2$ ، نخستین عضو permutation با احتمال مساوی هر عضو است. دو MinHash فقط وقتی برابرند که نخستین عضو از اشتراک آمده باشد؛ بنابراین احتمال $| اشتراک | / | اجتماع |$.

$$\hat{J}(C_1, C_2) = (\# \text{ equal Signature rows}) / K$$

$$E[\hat{J}] = J$$

هرچه تعداد توابع K بیشتر باشد، تخمین پایدارتر ولی حافظه/زمان بیشتر می‌شود.

Algorithm یک‌گذری MinHash بدون permutation

1. همه خانه‌های Signature را ∞ بگذار.
2. سطریهای Characteristic Matrix را یک‌بار scan کن.
3. اگر سند C در سطر j مقدار 1 داشت، برای هر تابع i بنویس: $\text{sig}(C)[i] = \min(\text{sig}(C)[i], h_i(j))$.

$$h_{a,b}(x) = ((a \cdot x + b) \bmod p) \bmod N \text{ (prime } p > N)$$

LSH: منطق AND/OR و S-Curve

- داخل هر band: AND روی r سطر؛ همه باید برابر باشند.
- بین bandها: OR؛ برابری فقط یک band کافی است.
- منحنی احتمال نسبت به s شکل S دارد؛ آستانه تقریبی $(1/r)^{(b/1)}$.
- r بیشتر: FP کمتر، FN بیشتر. b بیشتر: FN کمتر، FP بیشتر.
- Candidate بودن یعنی «ارزش بررسی دقیق دارد»، نه اینکه قطعاً مشابه است.
- Exact Check، FP را حذف می‌کند؛ FN از دست‌رفته را نمی‌تواند برگرداند.

False Negative = similar pair not selected as candidate

False Positive = dissimilar pair selected as candidate

Normalizing و Mixed Attributes

$$\text{Min-Max: } z_{if} = (x_{if} - \min(x_f)) / (\max(x_f) - \min(x_f))$$

$$\text{Z-score: } z_{if} = (x_{if} - \mu_f) / \sigma_f$$

$$\text{Mixed: } d(i, j) = \sum \delta_{ij}^f d_{ij}^f / \sum \delta_{ij}^f$$

$\delta=0$ اگر مقدار missing باشد یا صفت Binary نامتقارن و هر دو صفر باشند. Ordinal Attribute با M مرتبه ابتدا به $(M-1)/(rank-1)$ تبدیل و سپس مثل عددی محاسبه می‌شود.

هشدار: $\cosine-1$ یک dissimilarity رایج است، ولی لزوماً Metric نیست. Distance زاویه‌ای Metric $\arccos(\cosine)$ مناسب‌تری است. فرمول LCS فقط برای درج/حذف است، نه Levenshtein عمومی که جایگزینی هم دارد.

PCY – جزئیات Bitmap

1. Pass1: تک‌اقلام را دقیق بشمار و همه جفت‌های هر basket را به B bucket hash کن.

2. بین دو bucket: pass با $\text{count} \geq \text{minsup}$ را در bitmap علامت بزن.
 3. Pass2: جفت فقط اگر هر دو item frequent و bit=1 باشد شمارش شود.
 باکت frequent تضمین نمی‌کند خود جفت frequent باشد (collision)، اما باکت infrequent قطعاً نمی‌تواند جفت frequent داشته باشد.

PCY — Mini Example

minsup count=2 و {2,3} ، {1,2} ، {1,2,3} : Basket ها
 Pass 1: Item ها و Hash Bucket ها را Count کن.

count(1)=2, count(2)=3, count(3)=2 \n bucket h(1,2): count=2 -> frequent bit=1 \n bucket h(1,3): count=1 -> bit=0

Pass 2: Pair {1,3} به دلیل bit=0 حذف می‌شود، حتی اگر هر دو Item Frequent باشند.

Closed و Closure, Tidset

$$t(X) = \{ \text{tid} \in T \mid X \subseteq \text{transaction}(\text{tid}) \}$$

$$i(Y) = \{ \text{item} \in I \mid \text{item occurs in every tid} \in Y \}$$

$$\text{closure: } c(X) = i(t(X))$$

$$X \text{ is Closed} \Leftrightarrow c(X) = X$$

در کلاس itemset های با tidset یکسان، بزرگ‌ترین itemset همان Closed نماینده است. Closed یک نمایش lossless برای support است؛ Maximal فشرده‌تر است ولی support زیرمجموعه‌ها را از دست می‌دهد.

CHARM

- نمایش عمودی و itemset × tidset و prefix-equivalence class
- $t(X \cup Y) = t(X) \cap t(Y)$ = support اندازه این اشتراک
- تولید کاندید و شمارش support هم‌زمان است.
- tidset برابر $\rightarrow$ اقلام همیشه باهم‌اند؛ در closure ادغام شوند.
- رابطه زیرمجموعه tidset ها برای هرس شاخه‌ها استفاده می‌شود.

CHARM / Closure — Mini Example

Transactions: T1={A,B,C} , T2={A,B} , T3={A,B,C}

$$t(A) = \{1,2,3\} \setminus t(B) = \{1,2,3\} \setminus t(AB) = \{1,2,3\}$$

A و B چون Tidset یکسان دارند Merge می‌شوند. $c(A)=AB$ ؛ پس A Closed نیست.

قواعد Non-redundant

از Closed ها یک basis کوچک از قواعد می‌سازیم که سایر قواعد قابل استنتاج باشند. قواعد $\text{confidence}=100\%$ از closed بزرگ‌تر به کوچک‌ترند؛ قواعد کمتر از 100٪ به closed superset حرکت می‌کنند.

مثال‌های منبع

$$CW \Rightarrow C : \text{support}=5/6 , \text{confidence}=5/5=1 \setminus CW \Rightarrow ACTW : \text{support}=3/6=0.5 , \text{confidence}=3/5=0.6$$

Clustering

Definition: Unsupervised Learning برای تقسیم Object ها به Cluster هایی با Intra-cluster Similarity زیاد و Inter-cluster Similarity کم.

Purpose: کشف Structure پنهان وقتی Label از قبل وجود ندارد.

Execution: Stop Criterion و Representation, Similarity/Distance, Algorithm Family را انتخاب و نتیجه را Evaluate کن.

K-Means

Definition: Partitioning Algorithm که هر Cluster را با Centroid/Mean نمایش می‌دهد.

Purpose: Cluster Numeric Data, Spherical و اجرای سریع.

Execution: k Centroid اولیه → Assignment → به نزدیک‌ترین Repeat → Update Mean → Centroid تا Convergence.

DBSCAN

Definition: Density-based Clustering با دو Parameter به نام Eps و MinPts.

Purpose: کشف Shape دلخواه، Core/Border Point و Noise بدون تعیین k.

Execution: از یک Core شروع کن و همه Point های Density-reachable را Expand کن.

BFR

Definition: Scalable Clustering Algorithm برای Data عظیم با Summary های SUM, N, SUMSQ.

Purpose: وقتی Data در RAM جا نمی‌شود و Cluster ها تقریباً Gaussian و axis-aligned هستند.

Execution: Chunk ها را بخوان؛ Point ها را بین DS/CS/RS قرار بده و Summary ها را Update/Merge کن.

CURE

Definition: Hierarchical Clustering با چند Representative Point Shrink شده برای هر Cluster.

Purpose: Cluster های Non-spherical و مقاومت بهتر در برابر Outlier.

Execution: Hierarchical Clustering → Sample → انتخاب Representative های دور → Merge → Shrink/Assign.

OPTICS

Definition: Ordering Algorithm مبتنی بر Density که Reachability Structure را تولید می‌کند.

Purpose: Dataset هایی که Cluster های با Density متفاوت دارند.

Execution: Core Distance و Reachability Distance را حساب و Point ها را به ترتیب Priority پردازش کن.

هدف: اعضای یک Cluster به هم نزدیک؛ Clusters مختلف از هم دور. Unsupervised Learning (برخلاف Classification).
 انواع: Partitioning (K-Means, K-Medoids, PAM, CLARA, CLARANS) · Hierarchical (AGNES, DIANA) · Density: Grid (STING, CLIQUE) · (DBSCAN, OPTICS, DENCLUE)

مثال K-Means – فقط یک تکرار

نقاط: A(1,1) , B(1,2) , C(4,4) , D(5,4)

مراکز اولیه: $\mu_1 = A(1,1)$ و $\mu_2 = C(4,4)$

تخصیص: هر نقطه به نزدیکترین مرکز می‌رود.

A و B نزدیک μ_1 هستند $C_1 = \{A, B\}$

C و D نزدیک μ_2 هستند $C_2 = \{C, D\}$

به‌روزرسانی مرکز: میانگین بگیر.

$$\mu_1 = ((1+1)/2, (1+2)/2) = (1, 1.5)$$

$$\mu_2 = ((4+5)/2, (4+4)/2) = (4.5, 4)$$

پاسخ نهایی: مراکز جدید: (1.5, 1) و (4, 4.5)

Linkage: Single = کمینه Distance بین دو Cluster · Complete = میانگین · Centroid = Distance مراکز.

K-Medoids / PAM: نماینده Cluster یک نقطه واقعی است (مقاوم‌تر به پرت). CLARA روی نمونه کار می‌کند؛ CLARANS جستجوی تصادفی می‌کند.

$$\text{Core}(q) \Leftrightarrow |N_Eps(q)| \geq \text{MinPts}$$

$$\text{Border}(q) \Leftrightarrow \neg \text{Core}(q) \wedge \exists c: \text{Core}(c) \wedge q \in N_Eps(c)$$

$$\text{Noise}(q) \Leftrightarrow q \text{ is neither Core nor Border}$$

BFR Cluster Summary:

$$\text{centroid}_i = \text{SUM}_i / N$$

$$\text{variance}_i = \text{SUMSQ}_i / N - (\text{SUM}_i / N)^2$$

CURE: چند نقطه نماینده برای Clusters غیرکروی. **OPTICS**: وقتی Density Clusters فرق می‌کند، بهتر از DBSCAN است.

تکمله D – جزئیات کامل Algorithms Clustering [D4]

کاربردها و نیازها

کاربرد: بخش‌بندی مشتری، بازیابی اطلاعات، زیست‌شناسی، داده فضایی، امنیت، شناسایی outlier، پیش‌پردازش، جستجوی وب و خلاصه‌سازی.

نیازها: Scalability، انواع صفت، شکل دلخواه، دانش قبلی کم، تحمل Noise، افزایشی و مستقل از ترتیب، ابعاد بالا، محدودیت کاربر و تفسیرپذیری.

Hierarchical Clustering

روش	حرکت	شرح
AGNES	پایین→بالا	هر نقطه یک Cluster: نزدیک‌ترین‌ها ادغام
DIANA	بالا→پایین	همه یک Cluster: مرتب تقسیم

خروجی معمولاً **Dendrogram** است. سه تصمیم: نمایش Distance، دو Cluster، و شرط توقف.

$$\text{Centroid } \mu_C = (1/|C|) \sum_{x \in C} x$$

$$\text{Clustroid} = \operatorname{argmin}_{x \in C} \sum_{y \in C} d(x, y)$$

Centroid مناسب فضای اقلیدسی و ممکن است عضو واقعی داده نباشد. در اسناد/رشته‌ها «میانگین» معنی ندارد، پس clustroid یا medoid می‌گیریم.

Diameter و Linkage, Radius

$$\text{Single}(C_1, C_2) = \min d(x, y)$$

$$\text{Complete}(C_1, C_2) = \max d(x, y)$$

$$\text{Average}(C_1, C_2) = [\sum d(x, y)] / (|C_1||C_2|)$$

$$\text{radius}(C) = \max d(x, \mu_C)$$

$$\text{diameter}(C) = \max d(x, y)$$

- Single: مناسب شکل زنجیره‌ای، ولی اثر chaining و پل Noise دارد.
 - Complete: Cluster فشرده، ولی حساس به outlier.
 - Average: تعادل بین دو روش.
- توقف: رسیدن به k ، عبور radius/diameter از آستانه، افت Density، جهش کیفیت یا یک Cluster.

K-Means – نکات امتحانی

$$\text{SSE} = \sum_j \sum_{x \in C_j} \|x - \mu_j\|^2$$

- هر مرحله SSE را زیاد نمی‌کند، پس همگرا می‌شود؛ اما به بهینه محلی.
- مقداردهی بهتر: مراکز دور از هم، نمونه کوچک، یا چند اجرا و انتخاب کمترین SSE.
- انتخاب Elbow k : جایی که افزایش k دیگر بهبود زیاد نمی‌دهد.
- ضعف: نیاز به k ، حساس به init/outlier، نامناسب شکل غیرکروی و Nominal Attribute.

چرا Outlier بد است؟

میانگین نقاط $\{1, 2, 3\}$ با اضافه شدن 100: میانگین = 26.5

K-Medoids, PAM, CLARA, CLARANS

- **K-Medoids**: نماینده نقطه واقعی؛ Distance دلخواه؛ مقاوم‌تر ولی گران‌تر.
- **PAM**: k medoid → تخصیص → امتحان تعویض medoid/non-medoid → قبول اگر هزینه کم شد.
- **CLARA**: چند نمونه تصادفی، PAM روی هر نمونه، ارزیابی روی کل داده؛ سریع ولی وابسته به نمونه.
- **CLARANS**: جواب‌های medoid را گراف می‌بیند و همسایه تصادفی را می‌گردد؛ کیفیت بهتر، هنوز پرهزینه.

BFR – فرض‌ها و سه مجموعه

فضای اقلیدسی پُربعد؛ Cluster تقریباً نرمال؛ ابعاد مستقل؛ بیضی‌ها محور هم‌راستا. هر Cluster با $2d+1$ مقدار SUM، N ، خلاصه می‌شود.

معنی	مجموعه
با اطمینان عضو Cluster اصلی؛ خود نقاط دور ریخته و خلاصه حفظ می‌شود	DS (Discard Set)
گروه فشرده، ولی هنوز نزدیک هیچ DS نیست	CS (Compression Set)
نقاط منفرد/مشکوک که صریح نگه داشته می‌شوند	RS (Retained Set)

$$\text{Mahalanobis}(x,C) = \sqrt{\sum_i [(x_i - \mu_i) / \sigma_i]^2}$$

مراحل: نمونه اولیه و Cluster → RS → DS کردن RS و ساخت CS → خواندن chunk → نزدیک‌ها به DS، سپس CS، بقیه RS → ادغام CS‌ها → در chunk آخر ادغام مناسب با DS.

CURE

1. نمونه تصادفی در RAM
2. Hierarchical Clustering نمونه
3. انتخاب c نماینده که از هم دور باشند
4. حرکت هر نماینده کسری α به سمت centroid
5. Distance Clusters = کمترین Distance نماینده‌ها
6. تخصیص کل داده به نزدیک‌ترین نماینده

$$r' = r + \alpha(\mu - r)$$

نماینده‌های متعدد شکل Cluster را می‌گیرند؛ shrink به مرکز اثر outlier مرزی را کم می‌کند.

DBSCAN – روابط دقیق

$$N_{\text{Eps}}(q) = \{ p \mid \text{dist}(p,q) \leq \text{Eps} \}$$

$$\text{Core} \Leftrightarrow |N_{\text{Eps}}(q)| \geq \text{MinPts}$$

- **Directly density-reachable**: p در همسایگی q و q یک Core باشد؛ لزوماً متقارن نیست.
- **Density-reachable**: زنجیره‌ای از دسترسی‌های مستقیم.
- **Density-connected**: نقطه o وجود دارد که p و q هر دو از آن قابل دسترسی‌اند؛ متقارن است.
- با index فضایی تقریباً $O(n \log n)$ ، بدون آن $O(n^2)$.
- Eps کوچک → noise زیاد؛ Eps بزرگ → اتصال Clusters جدا.
- MinPts کم → Noise Cluster؛ MinPts زیاد → Cluster کوچک حذف.

OPTICS

$$\text{coreDist}(p) = \text{distance to the MinPts-th nearest neighbor}$$

$$\text{reachDist}(p,q) = \max(\text{coreDist}(q), \text{dist}(p,q))$$

به‌جای یک افراز، ترتیب نقاط و Reachability Plot می‌دهد. دره‌ها Cluster‌اند؛ دره عمیق‌تر یعنی Density بیشتر. مناسب‌تر از DBSCAN برای Density‌های متفاوت.

CLIQUE و STING

روش	ایده	مزیت	ضعف
STING	شبکه سلسله‌مراتبی؛ در هر سلول count/mean/min/max	سریع، موازی، افزایشی	مرز پله‌ای/محورهم‌راستا
CLIQUE	سلول‌های متراکم در زیرفضای پُر بعد؛ تولید زیرفضا با Apriori	کشف خودکار زیرفضا	حساس به اندازه grid و threshold

راهنمای انتخاب Algorithm

شرایط	انتخاب
عددی، کروی، سریع	K-Means
پرت زیاد یا Distance غیراقلیدسی	K-Medoids
داده عظیم و Cluster نرمال/محورهم راستا	BFR
شکل غیرکروی در فضای اقلیدسی	CURE
شکل دلخواه + Noise	DBSCAN
Density های متفاوت	OPTICS
ابعاد بالا و Cluster در بعضی ابعاد	CLIQUE

Execution و Mini Example برای Algorithm های فرعی

Algorithm	خلاصه Execution	Mini Example
AGNES	از Single Point ها شروع و نزدیک ترین Cluster ها را Merge کن.	اگر $d(A,B)=1$ کمترین باشد، اولین Merge برابر $\{A,B\}$ است.
DIANA	از یک Cluster کل شروع و دورترین Group را Split کن.	اگر D از بقیه بسیار دور باشد، نخست D جدا می شود.
PAM	Medoid/Non-medoid را Swap و فقط Cost بهتر را قبول کن.	اگر Swap از A به B، Cost را 8→12 کند، B Medoid جدید است.
CLARA	چند Sample بگیر، روی هر کدام PAM، سپس Cost را روی کل Data بسنج.	از یک میلیون Point پنج Sample و بهترین Medoid Set انتخاب می شود.
CLARANS	Neighbor تصادفی جواب Medoid را بررسی و به جواب بهتر Move کن.	Swap تصادفی $M1 \rightarrow P7$ اگر Cost کمتر کند پذیرفته می شود.
BFR	Point را با Mahalanobis به DS/CS بده؛ در غیر این صورت RS.	برای $(1,2), (3,4)$: $SUMS=(10,20)$, $SUM=(4,6)$, $N=2$, $\mu=(2,3)$.
CURE	Representative های دور را α به سمت Centroid Shrink کن.	$r=(2,0) \Rightarrow \alpha=0.2$, $\mu=(10,0)$, $r=(0,0)$.
DBSCAN	از Core شروع و Neighbor های Density-reachable را Expand کن.	Point: $Eps=1$, $MinPts=3$ با سه Neighbor، Core است.
OPTICS	Point ها را با Priority کمترین Reachability پردازش کن.	$coreDist(q)=2$, $dist(p,q)=3 \Rightarrow reachDist(p,q)=3$.
STING	از Cell سطح بالا، Cell نامرتبط را Prune و Child مرتبط را باز کن.	Spatial Query فقط Cell های intersecting را تا سطح پایین بررسی می کند.
CLIQUE	Dense Unit های 1D را با Apriori به Subspace بالاتر Expand کن.	Cluster ممکن است فقط در Subspace شامل Age و Income آشکار باشد.
DENCLUE	Kernel Density بساز و Point ها را به Density Attractor بفرست.	دو Peak جدا در Density Function دو Cluster ایجاد می کنند.

بخش 10 – حل کامل نمونه سوالات استاد [S10]

امتحان نمونه: ۹۰ دقیقه — ۱۰ نمره توصیفی

سوال 1 – تعاریف

الف) Min-hashing: از مجموعه بزرگ امضای کوتاه می سازیم طوری که احتمال برابر شدن hash دو مجموعه برابر Jaccard باشد.

$$\Pr[h(C_1) = h(C_2)] = J(C_1, C_2)$$

ب) Collaborative Filtering: Recommender System مبتنی بر Similarity کاربران یا آیتم ها (مثل Netflix/Amazon).

$$J(A,B) = |A \cap B| / |A \cup B|$$

د) Shingle: دنباله k توکن متوالی در سند.

سوال 2 – Cluster در برابر RDBMS

RDBMS: مقیاس غالباً عمودی، ACID، داده ساخت یافته، در حجم خیلی بالا گران/کند.
Cluster: مقیاس افقی، replication، commodity servers، failover، انتقال کد به سمت داده.

سوال 3 – Matrix×Vector با MapReduce

$$\text{Map: } (i, m_{ij} \times v_j)$$

$$\text{Reduce: } u_i = \sum_j (m_{ij} \times v_j)$$

مثال عددی کامل در بخش 5.

سوال 4 – TF-IDF

حل سوال استاد

$$TF = 10/100 = 0.1$$

$$IDF = \log_2(1024/128) = \log_2(8) = 3$$

$$TF-IDF = 0.1 \times 3 = 0.3$$

پاسخ نهایی: 0.3

سوال 5 – Apriori با MST = 0.3

TID	افلام	TID	افلام
1	a, b	5	a, b, c
2	b, c	6	d, f
3	a, b, c	7	c, d, e, f
4	d, e, f	8	a, b, c, d, e

حل قدم به قدم

تعداد = 8 Transaction و $\text{minsup} = 0.3 \rightarrow$ حداقل شمارش باید $3 \leq$ باشد (چون $2/8 = 0.25$ از 0.3 کمتر است).

Pass 1:

$a=4, b=5, c=5, d=4, e=3, f=3$

$L1 = \{a, b, c, d, e, f\} \rightarrow 3 \leq$ همه

Pass 2:

$ab=4 \checkmark \quad ac=3 \checkmark \quad bc=4 \checkmark$

$de=3 \checkmark \quad df=3 \checkmark$

$cd=2 \times \quad ce=2 \times \quad ef=2 \times$

$L2 = \{ab, ac, bc, de, df\}$

Pass 3: فقط abc قابل ساخت است (ab و ac و bc همه frequent اند). def نه، چون ef frequent نیست.

$\checkmark$ $\text{count}=3 \rightarrow 8$ و 5 Transactions در abc

$L3 = \{abc\}$

Pass 4 کاندیدی ندارد $\rightarrow$ توقف.

پاسخ نهایی: $L2 = \{ab, ac, bc, de, df\}$ و $L3 = \{abc\}$

چک لیست ۳۰ ثانیه‌ای: تعریف یک خطی؟ فرمول؟ مثال عددی؟ تفاوت با مفهوم نزدیک؟

موارد اضافه شده پس از مرور لکچرها: Hash/Index، دیسک، Data Streams، Bonferroni، Pregel، Multi-way Join، PCY، OPTICS/CURE/BFR. Online Advertising فقط در فهرست کلی overview بودند و اسلاید کامل در فایل‌های ارسالی نبود؛ تمرکز نمونه سوالات استاد روی Similar و Apriori و Items، MapReduce، TF-IDF، Cluster/RDBMS است.